实验三：软件实现与构建

一、根据 UML 图实现代码

1. UML 图到代码的转换

本次实现的是校园二手交易系统，使用 Python、Flask、Jinja 和 SQLite。实现时，先根据用例图划分注册登录、商品发布与审核、搜索、预约候补、成交确认、投诉申诉和管理统计等功能，再结合类图确定类与方法，依据状态图和时序图编写业务判断与调用流程。

类图与代码的主要对应关系如下。

UML 设计	代码实现	转换方式
User、Admin、InitialAdmin	`marketplace/domain.py`	保留用户、管理员和初始管理员的继承关系，通过角色判断操作权限
AccountService、EmailService	`marketplace/accounts.py`	将注册、登录、密码重置、验证码和会话管理转换为服务方法
Product、ProductVersion、ReviewRecord	`marketplace/schema.sql`、`marketplace/products.py`	用关系表保存商品、版本和审核记录，由 `ProductService` 完成保存、送审、审核和查询
Reservation、Waitlist	`marketplace/schema.sql`、`marketplace/trading.py`	用独立表保存预约与候补，由 `ReservationService` 处理预约、取消、候补转正和新价接受
Transaction、TransactionService	`marketplace/schema.sql`、`marketplace/trading.py`	保存成交记录，由 `TransactionService.confirm()` 处理双方成交确认
Complaint、Appeal、OperationLog	`marketplace/moderation.py`、`marketplace/schema.sql`	实现投诉申诉、账户和商品处置、分类统计及操作日志
Database	`marketplace/storage.py`	封装数据库连接、事务、查询和审计写入
WebUI	`marketplace/web.py`、`templates/`、`static/`	使用路由、页面模板和样式实现浏览器界面

对于需要持久化的实体，将属性转换为数据库字段，将实体关联转换为外键。例如，商品通过 `seller_id` 关联卖家，预约通过 `product_id` 和 `buyer_id` 关联商品与买家。商品内容保存为独立版本，审核记录对应具体版本，使修改后的内容重新接受审核。

根据商品状态图，在 `ProductStatus` 中定义草稿、待审核、审核驳回、已发布、已预约、已下架、已成交和违规封禁八种状态。状态转换由服务方法检查并执行。例如，预约成功后商品由 `PUBLISHED` 变为 `RESERVED`，双方确认成交后变为 `SOLD`，已成交商品不能取消交易或重新上架。

根据预约与候补时序图，将一次预约中的身份检查、商品状态检查、页面版本检查、预约记录创建和商品状态更新放在同一个数据库事务中。数据库部分唯一索引保证一件商品最多存在一条有效预约。卖家确认预约与确认成交分别记录，只有买卖双方都确认后才生成成交记录。候补按加入顺序处理，由卖家明确操作转正。

组件图中的分层落实为领域、应用服务、数据访问和页面模块。实验二设计中的 MySQL 在本次实现中调整为 SQLite，查询功能归入 `ProductService`，使用文件模块组织各层职责，并同步提供实验三 UML 图。

2. 软件源代码规模

代码内容	规模
`marketplace/` 下的 Python 实现	1667 行
自动化测试 Python 代码	538 行
验证与统计脚本 Python 代码	286 行
数据库建表文件	106 个物理行
HTML 页面模板	13 个文件

3. 使用大模型辅助代码实现

让大模型辅助完成的工作包括：

- 整理需求与 UML 的对应关系，划分模块，明确类、数据表和服务方法。
- 编写账户、商品、预约候补、成交、投诉申诉、统计日志等模块及页面。
- 将状态转换、角色权限、版本校验和事务规则落实到代码中。
- 编写自动化测试、浏览器检查、负载测试和源码统计脚本。
- 根据测试输出定位问题，修改代码，并使用 Ruff 检查代码规范和格式。

4. 生成内容不符合预期时的处理

对于不符合预期的内容，先指出与需求或运行结果不一致的地方，再补充相关文件、业务条件和期望结果，让模型针对具体问题修改。修改后重新执行对应检查，确认结果符合要求。通过补充上下文、明确规则、提供错误输出和重新验证，将模型生成的内容调整为符合项目要求的实现。

二、代码的编译与运行结果

1. 编译与代码检查

运行环境为 macOS、Apple M5、16 GiB 内存和 Python 3.13.3，项目使用虚拟环境管理依赖。Python 程序采用解释执行，使用 `compileall` 完成字节码编译检查。

在 `lab3/code` 目录执行：

```
.venv/bin/python -m compileall -q marketplace
.venv/bin/ruff check .
.venv/bin/ruff format --check .
.venv/bin/python -m pytest -q
```

结果

```
compileall：退出码 0，无错误输出
All checks passed!
18 files already formatted
24 passed in 8.33s
```

2. 启动与界面运行结果

项目提供一键启动脚本，完成环境安装和数据初始化后，在仓库根目录执行：

```
./start-lab3.sh
```

默认访问地址为 `http://127.0.0.1:5001`。本次另使用独立临时数据目录验证初始化和启动，数据库、管理员及演示数据均创建成功。在 5017 端口访问首页，返回 `HTTP 200`，响应页面包含“南大二手”商品列表，响应大小为 6441 字节，验证后停止服务。

已有浏览器验收记录显示，在 1366×768 桌面视口和 375×812 移动视口下，均完成了登录、查看详情、预约、取消、再次预约、卖家确认预约、私有留言和双方确认成交的流程。管理员的统计、商品管理和日志页面检查通过，JavaScript 错误记录为空。

3. 负载运行结果

根据已保存的本机 HTTPS 负载测试记录，初始商品数为 500，20 个并发用户持续运行约 300 秒，混合执行登录、列表、详情和预约请求。

指标	实测结果
请求样本数	5930
平均响应时间	1.014 秒
P95 响应时间	2.577 秒
预约成功请求	456
预约冲突等预期业务拒绝（HTTP 409）	732
非预期失败数	0

该次本机测试的 P95 响应时间小于 3 秒，非预期失败率为 0，达到设定的验收指标。

三、Git 远程代码管理展示

项目使用 Git 管理代码，远程托管在 GitHub，实验三使用 `lab3` 分支。

仓库地址：https://github.com/235826/secondhand-marketplace

分支地址：lab3 分支

开发过程按阶段保存提交，主要记录如下。

提交编号	提交内容
`cd143b7`	实现方案与验收计划
`5ef984c`	数据持久化与账户服务
`cd56048`	商品审核、预约候补与一致性交易
`a5dbbe5`	响应式页面与管理功能
`79ebb3d`	验收证据与实验材料
`4ddfba8`	一键启动脚本